

1 Create a sample spring boot application and configure H2 in memory database dependency.

```
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <!-- Source: https://mvnrepository.com/artifact/com.h2database/h2 -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <version>2.3.232</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

Added h2 dependency in pom.xml

2 Configure few properties under application properties file and read there value under Main class.

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RestDemoApplication implements CommandLineRunner{

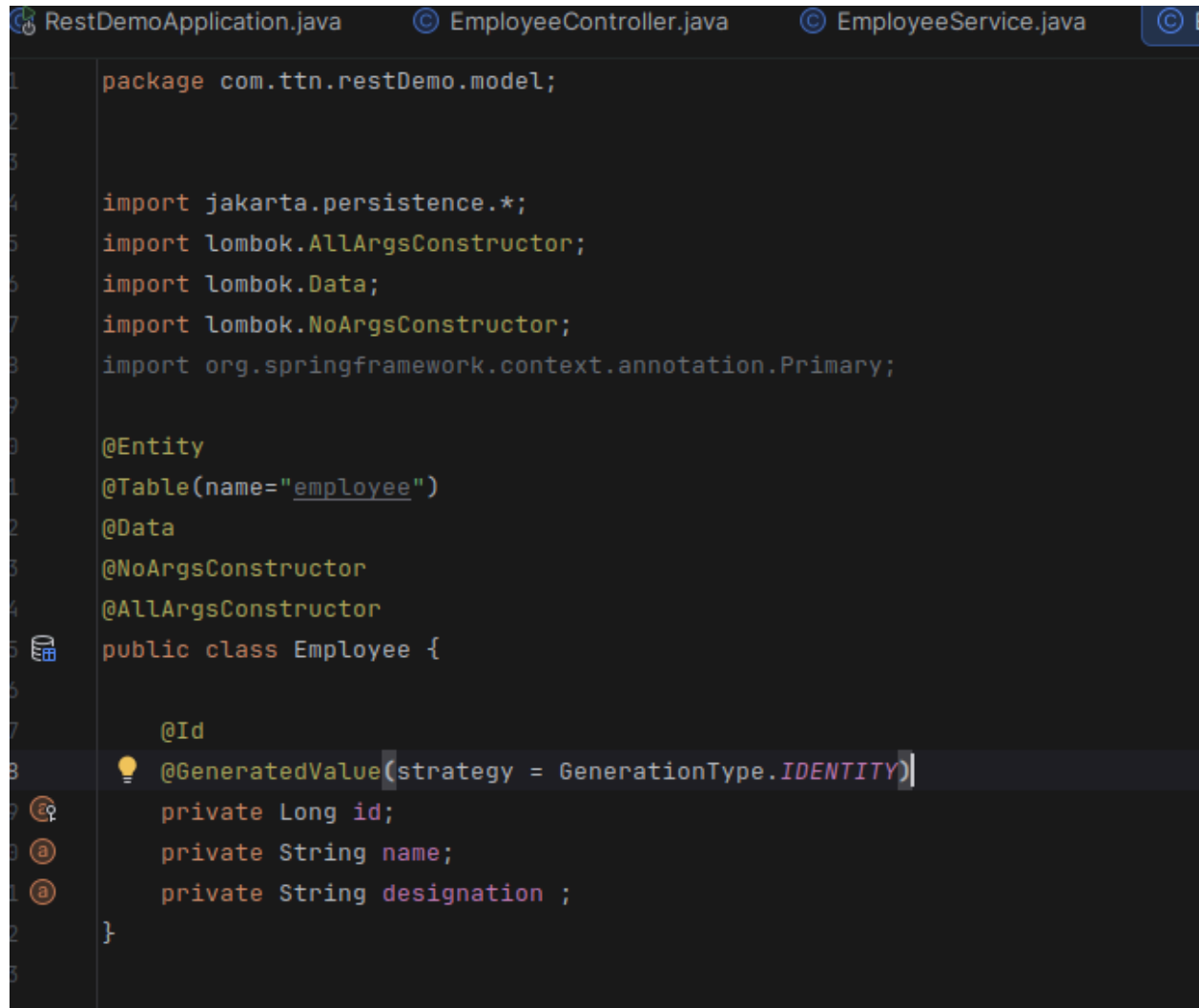
    @Value("${spring.application.name}")
    private String appName;

    @Value("${app.version}")
    private String appVersion;

    public static void main(String[] args) {
        SpringApplication.run(RestDemoApplication.class, args);
    }

    @Override
    public void run(String... args) {
        System.out.println("Application Name: " + appName);
        System.out.println("Application Version: " + appVersion);
    }
}
```

3) Create a Employee table (id, name, designation) under H2 database and inserts few sample records under that table at startup.



```
RestDemoApplication.java EmployeeController.java EmployeeService.java
package com.ttn.restDemo.model;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.context.annotation.Primary;

@Entity
@Table(name="employee")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String designation ;
}
```

This Is model class in which i used lombok dependency for constructor and getter and setter and for id auto generation

```
RestDemoApplication.java  EmployeeController.java  EmployeeService.java  Employee.java

package com.ttn.restDemo.repository;

import com.ttn.restDemo.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    // no usages
}
```

This Is repo layer using Jpa At Repo Layer

Created Controller With /getAllEmployee Mapping To Access data from database

```
package com.ttn.restDemo.controller;

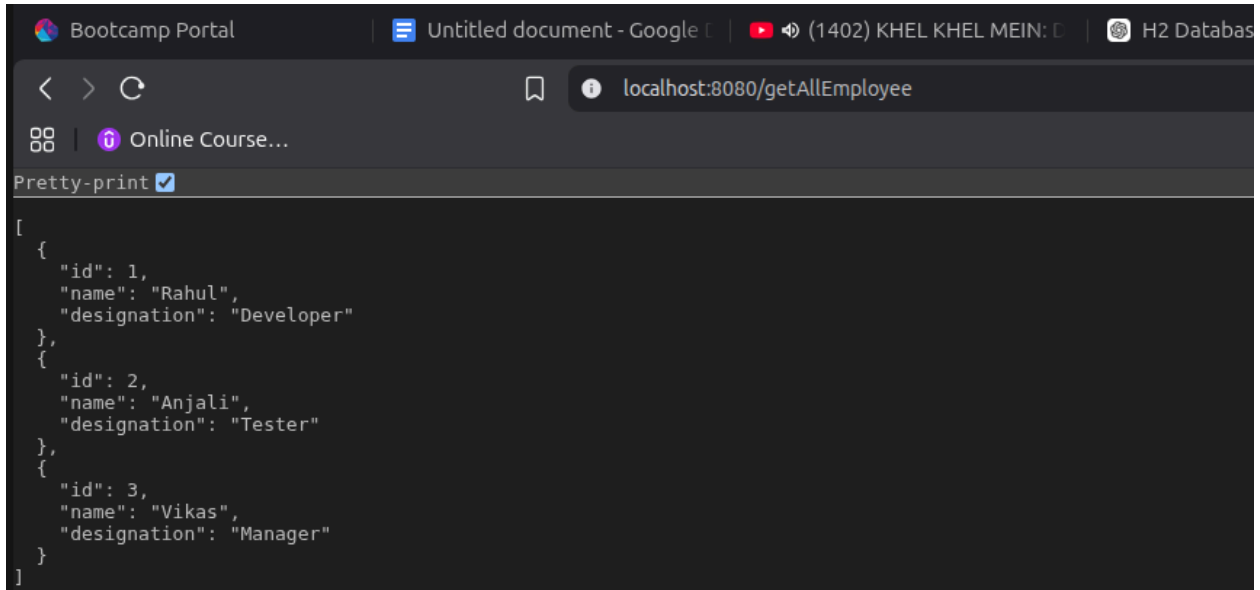
import com.ttn.restDemo.model.Employee;
import com.ttn.restDemo.service.EmployeeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class EmployeeController {

    @Autowired
    EmployeeService service;

    @GetMapping("/getAllEmployee")
    public List<Employee> GetAllEmployee(){
        return service.GetAllEmp();
    }
}
```



Bootcamp Portal | Untitled document - Google | (1402) KHEL KHEL MEIN: D | H2 Databases

localhost:8080/getAllEmployee

Online Course...

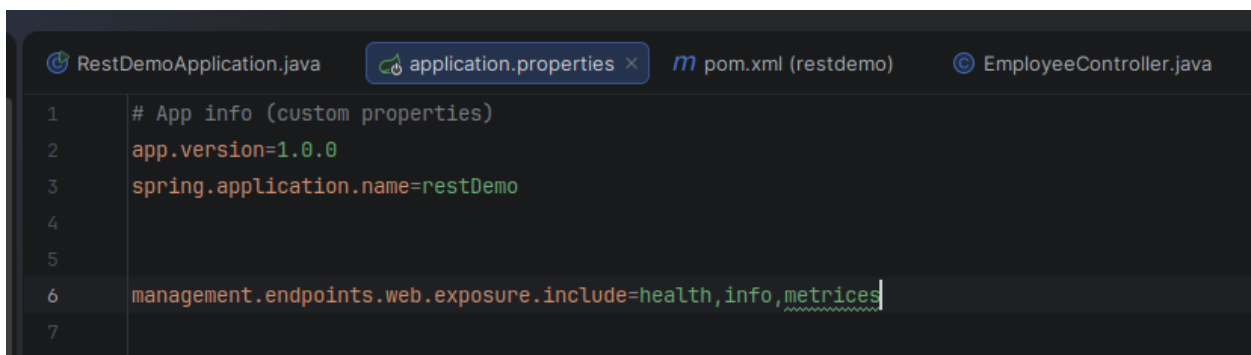
Pretty-print ☒

```
[
  {
    "id": 1,
    "name": "Rahul",
    "designation": "Developer"
  },
  {
    "id": 2,
    "name": "Anjali",
    "designation": "Tester"
  },
  {
    "id": 3,
    "name": "Vikas",
    "designation": "Manager"
  }
]
```

Testing Sample Data With url

URL– [“http://localhost:8080/getAllEmployee”](http://localhost:8080/getAllEmployee)

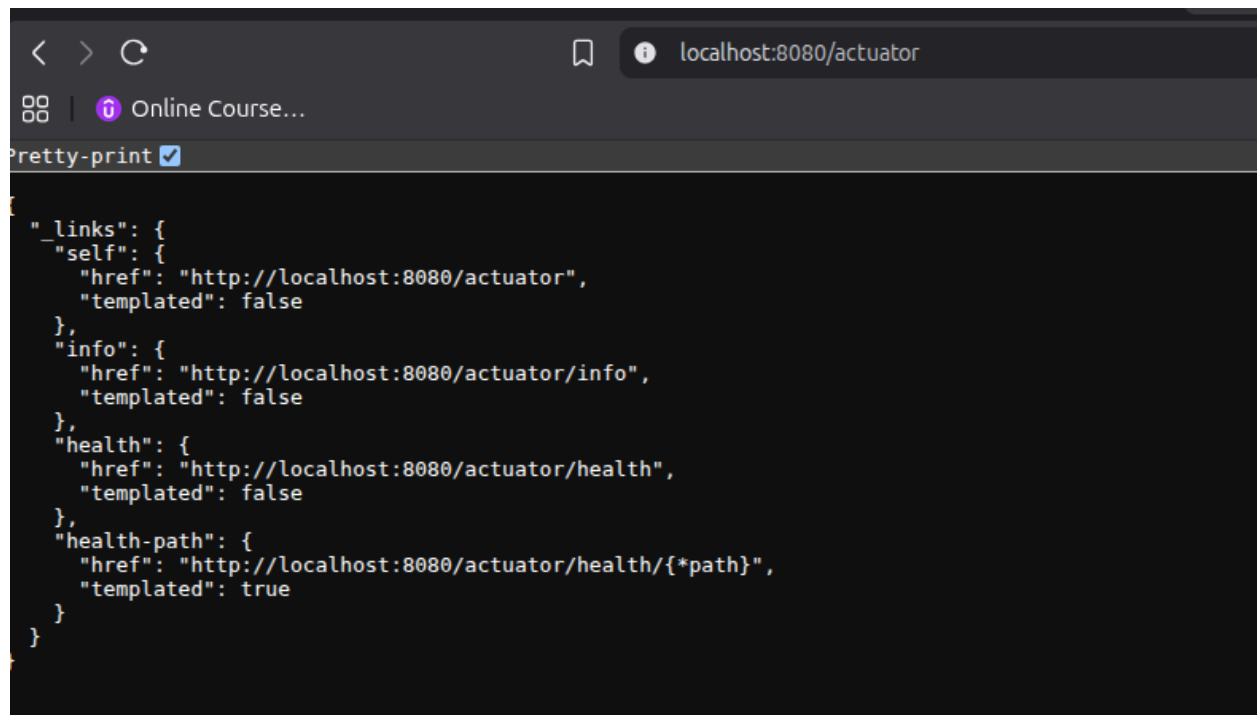
4Enable actuator services and play around few endpoints it exposes.



RestDemoApplication.java | application.properties x | pom.xml (restdemo) | EmployeeController.java

```
1 # App info (custom properties)
2 app.version=1.0.0
3 spring.application.name=restDemo
4
5
6 management.endpoints.web.exposure.include=health,info,metrics
7
```

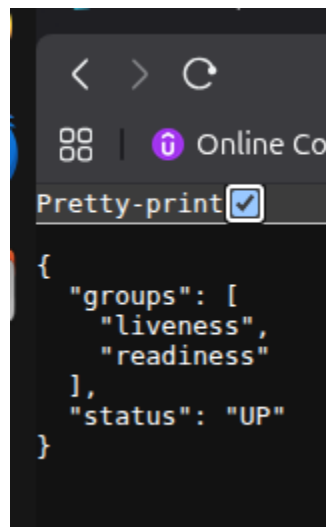
Exposed few Endpoints



A screenshot of a web browser window with the address bar showing `localhost:8080/actuator`. The browser's developer tools are open, displaying the response of the `/actuator` endpoint. The response is a JSON object with a `_links` property containing links to `self`, `info`, `health`, and `health-path`. The `health-path` link is templated and points to `http://localhost:8080/actuator/health/{*path}`.

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "info": {
      "href": "http://localhost:8080/actuator/info",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8080/actuator/health/{*path}",
      "templated": true
    }
  }
}
```

All Exposed End Points



A screenshot of a web browser window showing the response of the `/actuator/health` endpoint. The response is a JSON object with a `status` of `"UP"` and a `groups` array containing `"liveness"` and `"readiness"`.

```
{
  "groups": [
    "liveness",
    "readiness"
  ],
  "status": "UP"
}
```

Output of /actuator/health

Q)5) Enable devtools and play around by making changes.

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
```

Complex type: Depend
Description : The <den

Enabled devtools by adding dependency